



HASHDIT

HashDit

Smart Contract Security

ListaDao Lista Token LayerZero OFTV2 Contract Audit Report

ListaDao

26 June 2026 – 30 June 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	26 June 2026 -30 June 2026
Overall Risk	Medium
Project Name	ListaDao
Github	https://github.com/lista-dao/lista-token/pull/110
Branch	feat/lista-oft

Smart Contract List

No.	Contract Name	Verdict	Details
1	ListaOFTAdapterV2.sol	Informational	See Findings
2	ListaOFTv2.sol	Informational	See Findings
3	TransferLimiterV2.sol	Medium	See Findings
4	scripts/oft/v2	Informational	See Findings

Findings

[M01] Daily transfer limits in TransferLimiterV2 never reset if transfers are repeated within a 24 hour window

Contract TransferLimiterV2.sol

Severity Medium

Description

The TransferLimiterV2._checkAndUpdateTransferLimit() function is meant to enforce a per calendar day cap on cross chain transfers, as indicated in the code comments

However, it actually implements a sliding transfer limit based on the time since the last successful transfer rather than a fixed daily window, due to the lastUpdatedTime being updated whenever a transfer is successful

Description

JavaScript

```
function _checkAndUpdateTransferLimit(uint32 _dstEid, uint256 _amount,
address _user) internal virtual {
    TransferLimit memory limit = transferLimitConfigs[_dstEid];
    // check if transfer limit is set
    if (limit.dstEid == 0) {
        revert TransferLimitNotSet();
    }
    // check if amount is greater than 0
    if (_amount == 0) {
        revert TransferLimitExceeded();
    }

    // reset global transfer limit if the last transfer is made more
    than a calendar day
    if (isMoreThanACalendarDay(lastUpdatedTime[_dstEid],
block.timestamp)) {
        dailyTransferAmount[_dstEid] = 0;
    }
    // reset user transfer limit and attempt if the last transfer is
    made more than a calendar day
    if (isMoreThanACalendarDay(lastUserUpdatedTime[_dstEid][_user],
block.timestamp)) {
        userDailyTransferAmount[_dstEid][_user] = 0;
        userDailyAttempt[_dstEid][_user] = 0;
    }

    // check if the transfer amount exceeds the upper and lower limit
```

```

        if (_amount > limit.singleTransferUpperLimit || _amount <
limit.singleTransferLowerLimit) {
            revert TransferLimitExceeded();
        }

        // check if the transfer amount exceeds the daily transfer amount
limit
        if (dailyTransferAmount[_dstEid] + _amount >
limit.maxDailyTransferAmount) {
            revert TransferLimitExceeded();
        }

        // check if the transfer amount exceeds the daily transfer amount
limit per address
        if (userDailyTransferAmount[_dstEid][_user] + _amount >
limit.dailyTransferAmountPerAddress) {
            revert TransferLimitExceeded();
        }

        // check if the user exceeds the daily transfer attempt limit
        if (userDailyAttempt[_dstEid][_user] >=
limit.dailyTransferAttemptPerAddress) {
            revert TransferLimitExceeded();
        }

        // update global transfer limit and updated timestamp
        dailyTransferAmount[_dstEid] += _amount;
        lastUpdatedTime[_dstEid] = block.timestamp;
        // update user transfer limit and updated timestamp
        userDailyTransferAmount[_dstEid][_user] += _amount;
        userDailyAttempt[_dstEid][_user] += 1;
        lastUserUpdatedTime[_dstEid][_user] = block.timestamp;
    }
}

```

Impact

Because each successful transfer pushes the reference timestamp forward, the 24h transfer limit for each calendar day requirement is never satisfied as long as at least one transfer to that destination endpoint succeeds within any 24h period.

As a result, on a continuously active bridge the global counter is never reset and accumulates across many calendar days until it reaches maxDailyTransferAmount (1,000,000 LISTA). This results in the maxDailyTransferAmount cap behaving like a cumulative ceiling consumed in much less than a day rather than a daily allowance, producing outages that are unaligned with calendar days.

Note: The same flaw applies per address through lastUserUpdatedTime[_dstEid][_user], so userDailyTransferAmount (200,000 LISTA cap) and userDailyAttempt only reset after a personal 24h gap with zero successful transfers.

Recommendation

Consider resetting the counters based on a fixed epoch rather than the timestamp of the last successful transfer. Store a day index, for example `block.timestamp / 1 days`, alongside each counter, and when the current day index differs from the stored day index, zero `dailyTransferAmount[_dstEid]` (and the per address amount and attempt counters) and update the stored day index.

Fixed, now introduces the `_dayIndex` internal functions that computes the current rounded down day of the current `block.timestamp`, checking against both the `lastUpdatedTime` and `lastUserUpdatedTime`. The `isMoreThanACalendarDay` function has now been removed.

This ensures that the reset compares two fixed calendar buckets, and how recently `lastUpdatedTime` and `lastUserUpdatedTime` was written is irrelevant. All that matters is whether the last transfer and the current transfer land in the same UTC day. The moment `block.timestamp` crosses 00:00 UTC into a new bucket, increasing the computed `_dayIndex` by 1, the next transfer resets, even if the previous one was one second ago.

Note the following three caveats:

Status

1. Across a day boundary, the raw stored `dailyTransferAmount` and the sum of stored `userDailyTransferAmount` will not agree, wherein a global may already be zeroed for the new day while a user who hasn't transacted yet still carries the stale personal counter (reset is lazy, per-user). This is harmless on-chain (each stale value is reset-checked immediately before it's read, and nobody sums user counters against the global). It only affects an off-chain reader that trusts raw values, it must apply the same `_dayIndex` reset logic, exactly as with the lazy-reset caveat from before.
2. Noting the desync above, global cap can block a user with a "fresh" personal allowance, which is by design.. If other users have filled `dailyTransferAmount` today, a user whose personal counter just reset into a new day can still be rejected by the global cap check.
3. Because counters reset at 00:00 UTC regardless of recent activity, a user can move `dailyTransferAmountPerAddress` at 23:59:59 and again at 00:00:01, and the global `maxDailyTransferAmount` can likewise utilize up to 2x the daily transfer limit in a few seconds. For a rate-limiter whose purpose is to cap outflow during an incident, effective protection is halved at the day boundary. This is inherent to fixed-window limiting and is a deliberate tradeoff against the original's drifting window.

[M02] Disproportionate per-address transfer limit lets a few sybil addresses brick the bridge

Contract OFTScriptBase.sol

Severity **Medium**

Description

The transfer limiter applies three separate ceilings for each destination endpoint id: a global daily cap (maxDailyTransferAmount) that is shared by all users, a per-address daily amount cap (dailyTransferAmountPerAddress), and a per-address daily attempt cap (dailyTransferAttemptPerAddress). The global cap is a single shared bucket that is filled by the sum of every user's transfers.

Description

JavaScript

```
function _checkAndUpdateTransferLimit(uint32 _dstEid, uint256 _amount,
address _user) internal virtual {
    TransferLimit memory limit = transferLimitConfigs[_dstEid];
    // check if transfer limit is set
    if (limit.dstEid == 0) {
        revert TransferLimitNotSet();
    }
    // check if amount is greater than 0
    if (_amount == 0) {
        revert TransferLimitExceeded();
    }

    // reset global transfer limit if the last transfer is made more
    than a calendar day
    if (isMoreThanACalendarDay(lastUpdatedTime[_dstEid],
block.timestamp)) {
        dailyTransferAmount[_dstEid] = 0;
    }
    // reset user transfer limit and attempt if the last transfer is
    made more than a calendar day
    if (isMoreThanACalendarDay(lastUserUpdatedTime[_dstEid][_user],
block.timestamp)) {
        userDailyTransferAmount[_dstEid][_user] = 0;
        userDailyAttempt[_dstEid][_user] = 0;
    }

    // check if the transfer amount exceeds the upper and lower limit
    if (_amount > limit.singleTransferUpperLimit || _amount <
limit.singleTransferLowerLimit) {
        revert TransferLimitExceeded();
    }

    // check if the transfer amount exceeds the daily transfer amount
    limit
    if (dailyTransferAmount[_dstEid] + _amount >
limit.maxDailyTransfaerAmount) {
        revert TransferLimitExceeded();
    }
}
```



```

        // check if the transfer amount exceeds the daily transfer amount
        limit per address
        if (userDailyTransferAmount[_dstEid][_user] + _amount >
limit.dailyTransferAmountPerAddress) {
            revert TransferLimitExceeded();
        }

        // check if the user exceeds the daily transfer attempt limit
        if (userDailyAttempt[_dstEid][_user] >=
limit.dailyTransferAttemptPerAddress) {
            revert TransferLimitExceeded();
        }
        // update global transfer limit and updated timestamp
        dailyTransferAmount[_dstEid] += _amount;
        lastUpdatedTime[_dstEid] = block.timestamp;
        // update user transfer limit and updated timestamp
        userDailyTransferAmount[_dstEid][_user] += _amount;
        userDailyAttempt[_dstEid][_user] += 1;
        lastUserUpdatedTime[_dstEid][_user] = block.timestamp;
    }

```

In `OFTConfig._defaultLimits` the values are `maxDailyTransferAmount = 1,000,000`, `dailyTransferAmountPerAddress = 200,000`, `singleTransferUpperLimit = 100,000`, `singleTransferLowerLimit = 0.1`, and `dailyTransferAttemptPerAddress = 100`, all expressed in 18 decimal ether units. Because the per-address cap of 200,000 equals 20% of the global cap of 1,000,000, only 5 addresses are required to consume 100% of the global daily capacity, since $1,000,000 / 200,000 = 5$. The current attempt cap of 100 provides insufficient protection as with a single-transfer upper limit of 100,000, an address reaches its 200,000 per-address amount cap in just 2 transactions, using only 2 of its 100 allowed attempts.

JavaScript

```

function _defaultLimits(NetworkConfig memory cfg) private pure {
    cfg.maxDailyTransferAmount = 1_000_000 ether;
    cfg.singleTransferUpperLimit = 100_000 ether;
    cfg.singleTransferLowerLimit = 0.1 ether;
    cfg.dailyTransferAmountPerAddress = 200_000 ether;
    cfg.dailyTransferAttemptPerAddress = 100;
}

```

Impact

As a concrete example on the BSC to ETH route (dstEid 30101), an attacker controls 5 addresses, each holding 200,000 LISTA, for a total of 1,000,000 LISTA worth roughly \$48k at \$0.048 currently. Each address only need to bridge 100,000 Lista token twice.

After only 10 transactions across the 5 addresses, the global dailyTransferAmount reaches 1,000,000. Any honest user who then tries to bridge even the 0.1 minimum is reverted within the _setTransferLimitConfig check. The bridge direction is then unusable for all legitimate users.

The lockout can be both sustained and cheap based on current configurations. The global counter only resets after more than a calendar day has passed since lastUpdatedTime. Once the cap is reached, every further send reverts before lastUpdatedTime is updated, which freezes that timestamp and keeps the bridge locked for about 24 hours.

Per-address counters and attempts reset on the same rolling 24 hour basis, so the same 5 addresses can repeat the attack every day. The 1,000,000 LISTA principal is not lost, since it arrives on the destination chain and can be bridged back, which additionally grieves the reverse direction, so the ongoing cost is only gas plus LayerZero messaging fees for about 10 messages per day. No funds are lost, but available users cannot perform bridging when required

Consider the following configurations:

1. Set the per-address cap to roughly 1 to 2 percent of the global cap so that exhausting the shared bucket requires 50 to 100 distinct addresses rather than 5.
2. Account for the validation coupling in _setTransferLimitConfig, which requires $\text{dailyTransferAmountPerAddress} > \text{singleTransferUpperLimit}$ and $\text{maxDailyTransferAmount} > \text{singleTransferUpperLimit}$, so $\text{singleTransferUpperLimit}$ (currently 100,000) must be reduced at the same time. For example, use $\text{maxDailyTransferAmount} = 1,000,000$, $\text{dailyTransferAmountPerAddress} = 20,000$, $\text{singleTransferUpperLimit} = 10,000$, $\text{singleTransferLowerLimit} = 0.1$, and $\text{dailyTransferAttemptPerAddress} = 100$, which satisfies $1,000,000 > 20,000 > 10,000 > 0.1$.
3. Alternatively, raise the global cap, for instance to 25,000,000 (2.5%) with a per-address cap of 200,000 equal to 2%, if a 10,000 per-transaction ceiling is too restrictive. This would increase the current required funds to cause this permanent DoS to 25x of the current \$48K liquidity required, to around (\$1.2 million).

Recommendation

4. Lastly, consider an admin only function to clear dailyTransferAmount / per-user counters for an EID, so a limited channel (cap hit with no 24h idle gap) can be unstuck without waiting out the daily inactivity window.

Fixed, with the following caveats:

Status

1. The window fix does not remove the global-cap DoS, but it does make it materially more expensive, with one downside. This is the key point. The _dayIndex change relocated the reset to a fixed 00:00 UTC boundary. Crucially, that changes the sustain cost of the DoS:
2. There is still no manual recovery path, With the fixed window, a channel that hits its cap on honest volume self-heals only at the next 00:00 UTC, up to -24h away. There's no MANAGER/admin function to zero out daily transfer limits.
3. The config chose throughput over DoS-cost, which is an explicit tradeoff. Setting the global daily limit at 2M keeps the per-tx ceiling usable (30k) but still keeps the capital required to limit the bridge relatively low at (~\$100k), with 50 unique addresses. As such, continued monitoring should be performed, and if users abuse the limits to DoS the bridge, the admin can always retroactively increase limits via setTransferLimitConfigs.

[I01] Missing input validation in setPeer, setMsgInspector and setPreCrime allows zero configuration

Contract ListaOFTAdapterV2.sol & ListaOFTv2.sol

Severity Informational

Description

Currently, setPeer, setMsgInspector, and setPreCrime in both ListaOFTv2.sol and ListaOFTAdapterV2.sol perform no input validation.

Description

- setPeer accepts a bytes32 peer and writes it directly to the peer mapping. Setting the peer to bytes32(0) is the standard LayerZero mechanism for unsetting a peer and disabling messaging with that endpoint id, and the read path reverts only when a peer is queried while unset, so a non zero requirement here would remove the ability to retire or disable a channel.
- setMsgInspector accepts an address and stores it as the optional message inspector, where address(0) is the default and means no inspector is applied, so rejecting the zero address would make it impossible to turn the inspector off once set.
- setPreCrime accepts an address and stores it as the optional preCrime simulator, where address(0) likewise means no preCrime is configured, so rejecting zero would prevent disabling the simulator.

JavaScript

```
/// @notice Sets the trusted peer for a destination endpoint.
Restricted to MANAGER.
function setPeer(uint32 _eid, bytes32 _peer) public override
onlyRole(MANAGER) {
    _getOAppCoreStorage().peers[_eid] = _peer;
    emit PeerSet(_eid, _peer);
}

/// @notice Sets the message inspector. Restricted to MANAGER.
function setMsgInspector(address _msgInspector) public override
onlyRole(MANAGER) {
    _getOFTCoreStorage().msgInspector = _msgInspector;
    emit MsgInspectorSet(_msgInspector);
}

/// @notice Sets the preCrime contract. Restricted to MANAGER.
function setPreCrime(address _preCrime) public override
onlyRole(MANAGER) {
    _getOAppPreCrimeSimulatorStorage().preCrime = _preCrime;
    emit PreCrimeSet(_preCrime);
}
```

Impact

Because the function writes to storage directly and emits events, a MANAGER can accidentally/maliciously register a peer of zero or set a peer for an unintended endpoint id.

Recommendation

Given that the zero value is a valid and necessary configuration for unsetting a peer, disabling the message inspector, and disabling the preCrime simulator, ensure that the MANAGER enforces accurate inputs whenever such configurations are enabled.

Status

Acknowledged, it is presumed that the trusted MANAGER role will manage sensitive admin functionalities related to LayerZero bridge configurations appropriately.

[I02] Single transfer bound check runs after reset storage operations in _checkAndUpdateTransferLimit

Contract

TransferLimiterV2.sol

Severity

Informational

Description

Description

In TransferLimiterV2._checkAndUpdateTransferLimit(), the check that validates the transfer amount against the single transfer upper and lower bounds depends only on _amount and the in memory limit struct, so it has no dependency on the daily reset logic.

Despite this, it currently executes after the reset reads and writes for the global and per address counters. As a result, when a transfer amount is out of range, the function has already performed the reset SLOADs and any reset SSTOREs before reaching the bound check that reverts.

JavaScript

```
function _checkAndUpdateTransferLimit(uint32 _dstEid, uint256 _amount,
address _user) internal virtual {
    TransferLimit memory limit = transferLimitConfigs[_dstEid];
    // check if transfer limit is set
    if (limit.dstEid == 0) {
        revert TransferLimitNotSet();
    }
    // check if amount is greater than 0
    if (_amount == 0) {
        revert TransferLimitExceeded();
    }

    // reset global transfer limit if the last transfer is made more
    than a calendar day
    if (isMoreThanACalendarDay(lastUpdatedTime[_dstEid],
    block.timestamp)) {
        dailyTransferAmount[_dstEid] = 0;
    }
    // reset user transfer limit and attempt if the last transfer is
    made more than a calendar day
    if (isMoreThanACalendarDay(lastUserUpdatedTime[_dstEid][_user],
    block.timestamp)) {
        userDailyTransferAmount[_dstEid][_user] = 0;
        userDailyAttempt[_dstEid][_user] = 0;
    }

    // check if the transfer amount exceeds the upper and lower limit
    if (_amount > limit.singleTransferUpperLimit || _amount <
    limit.singleTransferLowerLimit) {
        revert TransferLimitExceeded();
    }
}
```

Impact

Because all gas consumed up to the point of a revert is still charged to the caller even though the state changes are rolled back, this ordering wastes gas on the revert path. Placing the bound check earlier would let an out of range amount revert before the function touches the lastUpdatedTime and lastUserUpdatedTime storage or executes any reset write. On the revert path this saves 2 to 4 cold SLOADs plus up to 3 SSTOREs of wasted work. The impact is limited to gas efficiency and does not affect correctness or security.

Recommendation	Consider moving the single transfer upper and lower bound check up so it runs immediately after the <code>_amount == 0</code> check, before the global and per address daily limit reset logic
Status	Fixed as recommended, shifts single transfer upper and lower bound check up so it runs immediately after the <code>_amount == 0</code> check, before the global and per address daily limit reset logic.

[I03] Missing lower bound validation on singleTransferLowerLimit in TransferLimiterV2

Contract	TransferLimiterV2.sol
Severity	Informational
Description	Description The TransferLimiterV2._setTransferLimitConfig() function validates the transfer limit configuration but does not require <code>singleTransferLowerLimit > 0</code>. The only existing constraint on this value is that <code>singleTransferUpperLimit > singleTransferLowerLimit</code>, which permits <code>singleTransferLowerLimit</code> to be configured as 0. It also does not align <code>singleTransferLowerLimit</code> with the OFT dust floor. <pre> JavaScript /** * @notice sets the transfer limit configurations * @param limit TransferLimit */ function _setTransferLimitConfig(TransferLimit memory limit) internal virtual { // validate transfer limit config require(limit.dstEid > 0, "dstEid must be greater than 0"); require(limit.singleTransferUpperLimit > limit.singleTransferLowerLimit, "upper limit must be greater than lower limit"); require(limit.dailyTransferAttemptPerAddress > 0, "dailyTransferAttemptPerAddress must be greater than 0"); require(limit.maxDailyTransferAmount > limit.singleTransferUpperLimit, "maxDailyTransferAmount must be greater than singleTransferUpperLimit"); require(limit.dailyTransferAmountPerAddress > limit.singleTransferUpperLimit, "dailyTransferAmountPerAddress must be greater than singleTransferUpperLimit"); require(</pre>

```

        limit.maxDailyTransferAmount >
limit.dailyTransferAmountPerAddress,
        "maxDailyTransferAmount must be greater than
dailyTransferAmountPerAddress"
    );
    // assign limit to the mapping
transferLimitConfigs[limit.dstEid] = limit;
    // emit event
emit TransferLimitChanged(
    limit.dstEid,
    limit.maxDailyTransferAmount,
    limit.singleTransferUpperLimit,
    limit.singleTransferLowerLimit,
    limit.dailyTransferAmountPerAddress,
    limit.dailyTransferAttemptPerAddress
);
}

```

Impact

Because localDecimals is 18 and sharedDecimals is 6, the decimalConversionRate is 10^{12} . Every bridged amount is rounded down to 6 decimal precision by `_removeDust` before the limiter evaluates it, and any value below 10^{12} wei (0.000001 LISTA) is truncated to 0. The truncation to zero case is already handled safely: `_debit` calls `_removeDust` before `_checkAndUpdateTransferLimit`, the limiter reverts when `_amount == 0`, and the configured `singleTransferLowerLimit` of 0.1 ether (1e17 wei) is 100000 times the 10^{12} dust floor. As a result no realistic transfer can be fully truncated to zero, and this is a configuration hardening concern rather than an exploitable bug.

Consider adding a require check that `singleTransferLowerLimit > 0` in `_setTransferLimitConfig()` to document intent and reject a meaningless zero lower bound. This also has an additional side effect that ensures that all other daily limits are non-zero as well.

Recommendation

Note that applying this lower bound check effectively makes it consistent that both a per user and global daily limit must always be enforced within the TransferLimiter and cannot be disabled by zeroing out the limit (the only exception is to set a low limits, e.g. wei levels)

Status

Acknowledged, it is presumed that the trusted MANAGER role and/or deployer deploying the contract will not input a `singleTransferLowerLimit` that is equal to zero.

[I04] setPeer trust anchor should be gated by the DEFAULT_ADMIN_ROLE (Timelock) instead of MANAGER

Contract ListaOFTAdapterV2.sol & ListaOFTv2.sol

Severity Informational

Description	Description In ListaOFTAdapter, the setPeer function is currently restricted to the MANAGER role. This function defines the trusted peer for a given endpoint ID, which is the core trust anchor of the cross-chain bridge. <pre> JavaScript /// @notice Sets the trusted peer for a destination endpoint. Restricted to MANAGER. function setPeer(uint32 _eid, bytes32 _peer) public override onlyRole(MANAGER) { _getOAppCoreStorage().peers[_eid] = _peer; emit PeerSet(_eid, _peer); } </pre>
Recommendation	Impact Any message arriving from a configured peer is treated as authenticated. If a malicious or incorrect peer is set on a configured EID, an attacker can deliver forged messages that the contract will accept as legitimate, leading to minting of unbacked tokens or unlocking of escrowed funds (see the 2025 GriffinAI hack) It is rare to need a peer change, and it is not an emergency operation, since the pause mechanism already covers the case of severing a compromised remote quickly to zero, so this is an access control improvement suggestion. Consider restricting the setPeer function to the DEFAULT_ADMIN_ROLE instead of the MANAGER role, in the off-chance that 3 of the 6 required private keys are compromised.
Status	Fixed, setPeer is not gated to the DEFAULT_ADMIN_ROLE which is a Timelock contract with a minimum 1 day delay.

[I05] Inconsistent __gap size in TransferLimiterV2 breaks the 50-slot gap convention	
Contract	TransferLimiterV2
Severity	Informational
Description	Description TransferLimiterV2 defines 6 storage mappings (transferLimitConfigs, dailyTransferAmount, lastUpdatedTime, userDailyTransferAmount, userDailyAttempt, and lastUserUpdatedTime), which take up 6 storage slots, and then declares a uint256[45] storage gap. The total comes to 51 slots, which is one slot more than the 50-slot reserved-window convention used everywhere else in the codebase. OpenZeppelin contracts-upgradeable 4.9.3 follows the rule of used + gap = 50, and the sibling

contracts ListaOFTv2 and ListaOFTAdapterV2 correctly apply it with 0 used + gap[50] = 50.

```
JavaScript
/**
 * @dev Storage gap to allow new state variables to be added to this
base
 * contract in future upgrades without shifting the storage
layout of
 * inheriting contracts.
 */
uint256[45] private __gap;
```

Impact

This is not a live storage collision today, since the extra gap is simply empty reserved space. The problem is that the mismatch makes the contract inconsistent with both the OpenZeppelin convention and its sibling contracts, so a maintainer who assumes gap = 50 - used will possibly size the gap incorrectly in future upgrades

Recommendation

Consider adjusting the __gap declaration from uint256[45] to uint256[44] for the TransferLimiter contract so that all three contracts use uniform 50-slot windows.

Status

Fixed, now reduces gap size to uint256[44] to follow the 50 slot convention.

[I06] Missing inbound transfer limit on the _credit path leaves cross-chain minting uncapped

Contract ListaOFTAdapterV2.sol & ListaOFTv2.sol

Severity Informational

Description

Description

The TransferLimiterV2._checkAndUpdateTransferLimit() function is only invoked on the _debit path of ListaOFTv2 and ListaOFTAdapterV2, which is the outbound (send) side on the source chain. The inbound _credit path, which mints tokens in ListaOFTv2 and unlocks escrowed tokens in ListaOFTAdapterV2, applies no equivalent limit.

```
JavaScript
/**
 * @dev Mints tokens to the recipient when receiving cross-chain.
 * Blocked while the contract is paused.
 */
function _credit(
    address _to,
    uint256 _amountLD,
    uint32 _srcEid
) internal virtual override whenNotPaused returns (uint256
amountReceivedLD) {
    return super._credit(_to, _amountLD, _srcEid);
```

```
}
```

Impact

The debit side limit only constrains abuse that originates on the source chain, so it provides no protection against a compromised or buggy peer, a bridge or DVN fault, or a key compromise on another chain. Any of those can deliver an inflated inbound message, and the local chain will mint or unlock the full amount with no daily ceiling.

Since `_credit` is what actually creates supply on this chain, the absence of a cap there leaves the most security relevant path unguarded, even though the outbound path is rate limited.

Recommendation

Consider an inbound limit on `_credit` using a purpose built rate limiter rather than extending `TransferLimiterV2`. However, this might be too complex, given `_credit` executes inside `IzReceive`, so a hard revert moves the message into the stored and retryable state and, due to nonce ordering, can stall the entire inbound channel from that peer until the cap resets or an owner intervenes.

Instead, off-chain security enforcements should be performed to ensure that no exploitation or malicious configuration of target chain bridge parameters and keys are possible.

Status

Acknowledged, it is presumed that sufficient off-chain security enforcements and monitoring will be performed to ensure that no exploitation or malicious configuration of target chain bridge parameters and keys are possible.

[I07] Design considerations between `TransferLimiterV2` and `RateLimiter`

Contract `ListaOFTAdapterV2.sol` & `ListaOFTv2.sol`

Severity **Informational**

Description

Description

The project currently has two rate limiting contracts within the `oft/v2` folder,, and the choice between them matters because each is suited to a different transfer direction, while the inbound path is left unprotected.

`RateLimiter` is already present in the `v2` folder and is a fork of LayerZero's [oapp/utlis/RateLimiter](#), yet it is never inherited or used. `TransferLimiterV2` is the contract actually wired into the `_debit` path of `ListaOFTv2` and `ListaOFTAdapterV2`.

The two contracts model a limit differently.

- `TransferLimiterV2` resets on a calendar day boundary computed by `isMoreThanACalendarDay`, and it tracks a global per endpoint id daily cap, a per

address daily cap, a per transaction upper and lower bound, and a per address daily attempt counter. Its day boundary is anchored to the timestamp of the last transfer rather than to a fixed calendar day, so the reset point drifts, and because the full quota becomes available the instant the boundary is crossed, transfers structured around that edge can release two full daily caps in quick succession. It also validates its configuration extensively in `_setTransferLimitConfig` and carries a storage gap for proxy upgrades.

- RateLimiter instead uses a continuous linear decay over a configurable window through `_checkAndUpdateRateLimit` and `_amountCanBeSent`, exposes only a single global per endpoint id cap, performs no field validation, and has no storage gap.
- On the outbound `_debit` path, the limited address is `_from`, which is the real payer, so the per address caps, per transaction bounds, and attempt counter of TransferLimiterV2 are more secure and should be kept. We can consider replacing its calendar day reset with the windowed decay model of RateLimiter to remove the drifting boundary and the double quota release

If more flexibility is required, considering adopting a linearly decaying model of RateLimiter to remove the immediate limit reset during a daily calendar day pass

If not, if a refactor of the TransferLimiter contract is too complex, we should instead routinely monitor and set conservative configurations

- Start conservative on every `dstEid` and raise limits as confidence and volume grow.
- Consider an admin only function to clear `dailyTransferAmount` / per-user counters for an EID, so a wedged channel (cap hit with no 24h idle gap) can be unstuck without waiting out the inactivity window, as mentioned in [M02]
- Monitor utilization per EID and alert as `dailyTransferAmount` approaches `maxDailyTransferAmount`, so you can intervene before legitimate traffic starts reverting.
- Treat all of this as outbound-only mitigation. It does nothing for the inbound `_credit` exposure, so the manual pause remains your only inbound circuit breaker until an inbound limiter is added.

Recommendation

Status

Acknowledged, the fixed window transfer limiter is presumed to be the accepted design choice.

[I08] isMoreThanACalendarDay measures elapsed seconds rather than calendar days and lacks input validation**Contract**

TransferLimiterV2.sol

Severity

Informational

Description

The TransferLimiterV2.isMoreThanACalendarDay() function does not perform any calendar day computation. It computes $(\text{timestampB} - \text{timestampA}) / 86400$ and returns whether the result is ≥ 1 , which is purely a check for whether at least 86400 seconds have elapsed.

JavaScript

```
/**
 * @notice compare two timestamp and check if the difference is more
 * than a calendar day
 * @param timestampA timestamp A
 * @param timestampB timestamp B
 * @return true if the difference is more than a calendar day
 */
function isMoreThanACalendarDay(uint256 timestampA, uint256
timestampB) internal pure virtual returns (bool) {
    uint256 secondsPerDay = 86400; // 60 * 60 * 24
    uint256 diffInDays = (timestampB - timestampA) / secondsPerDay;
    return diffInDays >= 1;
}
```

Description**Impact**

The issues that do matter are different from what the name suggests.

1. First, the name is misleading and this is the substantive concern. "Calendar day" implies a reset at a fixed boundary such as 00:00 UTC, but the logic is a rolling 24 hour window anchored to the last transfer's timestamp. Because lastUpdatedTime and lastUserUpdatedTime are rewritten on every transfer, the window only resets after 24h of complete inactivity and never at a fixed daily point, whereas a true calendar day reset would zero the counter at midnight UTC regardless of traffic.
2. Second, the subtraction $\text{timestampB} - \text{timestampA}$ reverts under Solidity 0.8 checked math if $\text{timestampA} > \text{timestampB}$. In practice it is safe because timestampA is a stored past block.timestamp and timestampB is the current one, which is monotonically non decreasing, and the fresh state value of 0

simply triggers a harmless reset on an already empty bucket. It is thus only an issue if the function is called directly or by integrators with wrong timestamp inputs.

3. Third, the floor division creates a boundary at exactly 86400 seconds, so at 86399 seconds the function returns false, meaning the behavior is really ≥ 1 day rather than strictly "more than a calendar day", and this off by the boundary semantics is worth stating precisely.

Based on issue [\[M01\]](#), it is presumed that a rolling elapsed time window is NOT the intended behavior, given it would permanently accumulate transfer limits, so the current function name and logic would require only slight adjustments.

Recommendation

Assuming a fix is performed for [\[M01\]](#), document the ≥ 86400 boundary and the assumption that the stored timestamp is always passed as the first argument, and consider renaming `isMoreThanACalendarDay` to `isMoreThanEqualToACalendarDay` to improve naming semantics.

Status

Fixed, the `isMoreThanACalendarDay` has now been removed and replaced with the `_dayIndex` function.

[I09] Zero transfer amount reverts with the misleading `TransferLimitExceeded` error

Contract

`TransferLimiterV2.sol`

Severity

Informational

Description

In `TransferLimiterV2._checkAndUpdateTransferLimit`, a transfer amount of 0 is rejected by reverting with `TransferLimitExceeded`. This error name is misleading, because a zero amount is not a case of the transfer exceeding any configured limit. The zero value often arises when `_removeDust` reduces a sub `1e12` input down to 0, so the caller is notified that they exceeded a limit when the real reason is that the amount rounded to zero.

Description

JavaScript

```
function _checkAndUpdateTransferLimit(uint32 _dstEid, uint256 _amount,
address _user) internal virtual {
    TransferLimit memory limit = transferLimitConfigs[_dstEid];
    // check if transfer limit is set
    if (limit.dstEid == 0) {
        revert TransferLimitNotSet();
    }
    // check if amount is greater than 0
    if (_amount == 0) {
        revert TransferLimitExceeded();
    }
}
```

```
}
```

Impact

The mismatch makes diagnosing failed transfers harder, since the revert reason points operators and integrators toward limit configuration rather than toward the ac

Recommendation

Consider introducing a dedicated ZeroAmount() error and revert with it when `_amount == 0`, leaving TransferLimitExceeded for the genuine cases where an amount breaches a configured limit.

Status

Fixed, a dedicated ZeroAmount() error has been introduced and is invoked when a zero amount is utilized during bridging actions.

[I10] Route disablement via setPeer is symmetric only and can strand in-flight inbound transfers**Contract**

TransferLimiterV2.sol

Severity

Informational

Description**Description**

Currently, transfer limiter specific routes cannot be disabled due to the following:

1. The `dstEid > 0` requirement and non-zero checks within `_setTransferLimitConfig` enforces non-zero limits once a transfer limiter configuration is set for a specific `dstEid`. This results in a possible dangling configuration cannot be removed.
2. Second, `setPeer(eid, 0)` disables both directions at once, and doing so while messages are in flight can strand inbound transfers. There is no mode that stops new outbound transfers while still allowing already sent inbound messages to settle. If the peer is zeroed while messages from the remote chain are in flight, those inbound messages then fail `_getPeerOrRevert` at `IzReceive` and become stuck. They are retryable once a peer is restored, and the value has already been burned or locked on the source side, so the funds are liveness stranded rather than lost.

Recommendation

Consider documenting the dual nature of `setPeer(eid, 0)`, making clear that it both disables a route and, if used while inbound messages are in flight, can strand those messages until a peer is restored, and that pause shares the same both directions behavior.

Status

Acknowledged, it is presumed that sufficient off-chain security enforcements and monitoring will be performed to ensure that no exploitation or malicious configuration of target chain bridge parameters and keys are possible on both the source chain and the target chain.

[I11] renounceOwnership() permanently freezes the LayerZero delegate and removes all ability to rotate it

Contract ListaOFTAdapterV2.sol & ListaOFTv2.sol

Severity Informational

Description

In LayerZero V2, the delegate is an address authorized to configure the OApp's messaging settings directly on the LayerZero EndpointV2. The delegate can call endpoint.setConfig to set the ULN and DVN configuration (which DVNs verify, the required and optional counts, the optionalDVNThreshold, the confirmations, and the executor settings), can call setSendLibrary, setReceiveLibrary, and setReceiveLibraryTimeout to choose the messaging libraries for the route, and can call skip, clear, nullify, and burn to manage the inbound message pipeline, including dropping or unblocking stuck payloads. The delegate therefore controls the OApp's verification security stack and its inbound message lifecycle. The only function that can change who the delegate is is setDelegate, which is owner gated and inherited from OAppCoreUpgradeable, and initialize sets the delegate to the MANAGER.

Description

JavaScript

```
/**
 * @dev Leaves the contract without owner. It will not be possible
 * to call
 * `onlyOwner` functions. Can only be called by the current owner.
 *
 * NOTE: Renouncing ownership will leave the contract without an
 * owner,
 * thereby disabling any functionality that is only available to the
 * owner.
 */
function renounceOwnership() public virtual onlyOwner {
    _transferOwnership(address(0));
}
```

Impact

OwnableUpgradeable.renounceOwnership() is inherited and gated by onlyOwner, and it is not overridden. The only power the owner actively exercises in these contracts is setDelegate, because every other configuration action was moved to AccessControl roles. If the admin renounces ownership, which is a realistic action when decentralizing or dropping the owner key since roles appear to be the intended control plane, the delegate becomes permanently unchangeable, as no address can call setDelegate anymore. Once ownership is renounced, the delegate is frozen forever as whatever address it currently holds, whether the MANAGER set at initialization or a value set later. T

Recommendation	Consider overriding the <code>renounceOwnership()</code> so that it reverts, ensuring ownership cannot be dropped while the ability to rotate the delegate must be preserved.
Status	Acknowledged, it is presumed that the trusted <code>MANAGER</code> role will not perform malicious/accidental <code>renounceOwnership</code> calls.

[I12] EIP-2612 permit on ListaOFTv2 can be front-run to grief dependent actions

Contract	ListaOFTAdapterV2.sol & ListaOFTv2.sol
Severity	Informational

Description	Description ListaOFTv2 inherits the standard EIP-2612 permit function from OpenZeppelin's ERC20PermitUpgradeable. This function is callable by anyone, because it only requires a valid signature and does not require that the caller is the signer. As a result, any observer of a pending permit transaction in the mempool can copy the signature and submit it first. The front-running permit call succeeds, sets the intended approval, and increments the owner's nonce. When the victim's original permit transaction is then mined, it reverts, because the nonce it was signed against has already been used and the recovered signer no longer matches. <pre> JavaScript /** * @dev See {IERC20Permit-permit}. */ function permit(address owner, address spender, uint256 value, uint256 deadline, uint8 v, bytes32 r, bytes32 s) public virtual override { require(block.timestamp <= deadline, "ERC20Permit: expired deadline"); bytes32 structHash = keccak256(abi.encode(_PERMIT_TYPEHASH, owner, spender, value, _useNonce(owner), deadline)); bytes32 hash = _hashTypedDataV4(structHash); address signer = ECDSAUpgradeable.recover(hash, v, r, s); require(signer == owner, "ERC20Permit: invalid signature"); _approve(owner, spender, value); } </pre>
--------------------	--

Impact

This is the known EIP-2612 front-running and griefing behavior and the impact is limited, since the approval the victim wanted is still established by the front-runner's call, so there is no theft and no loss of funds.

If the victim bundles a permit with a follow-up action in a single transaction, which is the common pattern such as permit followed by transferFrom or permit followed by a bridge send, that entire transaction reverts on the already-consumed nonce and the victim has to resubmit, causing a DoS.

This behavior is inherent to EIP-2612 and is not specific to this contract, so no on-chain change is strictly required.

Document it for integrators and front-end developers. Integrations should make permit usage resilient, for example by wrapping the permit call in a try/catch and continuing if it reverts, or by checking the current allowance first and skipping permit when the allowance is already sufficient, so that a front-run permit does not break the dependent action.

Note that the adapter side bridge send path uses approve and transferFrom rather than permit, so this only affects the ListaOFTv2 token's permit convenience function and any integrations that depend on it.

Recommendation

Status

Acknowledged, it is presumed that any integrators of Lista LZ supported permit functionality utilize necessary try/catch if required.

[I13] Deployment roles silently default to the deployer EOA in OFTScriptBase._roles

Contract

OFTScriptBase.sol

Severity

Informational

Description

The OFTScriptBase._roles() function resolves the ADMIN, MANAGER, and PAUSER addresses from environment variables, and each one falls back to the deployer address when its variable is not set. If an operator runs either deploy script without explicitly setting ADMIN, MANAGER, and PAUSER, all three roles are granted to a single deployer EOA. This happens with no warning and no revert.

Description

JavaScript

```
function _roles(address deployer) internal view returns (address admin,
address manager, address pauser) {
    admin = vm.envOr("ADMIN", deployer);
    manager = vm.envOr("MANAGER", deployer);
    pauser = vm.envOr("PAUSER", deployer);
}
```

	Impact This provides a small time window where each sensitive admin role is owned by a single exploitable private key based on the deployer EOA.
Recommendation	Consider including necessary role revocation and role granting logic within the DeployListOFTAdapterV2.s.sol and DeployListOFTv2.s.sol to ensure this is performed atomically during deployment.
Status	Fixed, now includes a warning within the OFTScriptBase.sol base script if any sensitive roles is resolved to the deployer EOA. It is presumed that the roles will be revoked and granted appropriately in separate transactions shortly after deployment,

[I14] Mainnet DVN policy uses zero required DVNs

Contract	OFTConfig.sol & SetDVNConfig.s.sol
Severity	Informational
Description	Description The mainnet DVN security policy configured for both the BSC adapter and the Ethereum OFT uses zero required DVNs and relies entirely on an optional DVN threshold of 3-of-4. <pre>JavaScript cfg.requiredDVNs = new address[](0); cfg.optionalDVNs = new address[] (4);</pre> SetDVNConfig.s.sol encodes this through _dvnCount, which returns NIL_DVN_COUNT (type(uint8).max, 0xff) when the DVN list is empty. The encoding is correct. In the LayerZero ULN, a requiredDVNCount of 0xff (NIL) explicitly means no required DVNs and also overrides away any default required DVN the pathway would otherwise inherit, whereas a value of 0 (DEFAULT) would inherit the chain default. The endpoint accepts the config because the ULN only requires that requiredDVNCount > 0 OR optionalDVNThreshold > 0. <pre>JavaScript function _dvnCount(address[] memory dvns) internal pure returns (uint8) { return dvns.length == 0 ? NIL_DVN_COUNT : uint8(dvns.length); }</pre> Impact

The receive side verification in the LayerZero ULN, `_checkVerifiable`, enforces ALL required DVNs as an AND gate plus a threshold M-of-N of the optional DVNs. With zero required DVNs the required loop is skipped entirely, so an inbound message is committed and then executed, minting on the OFT and unlocking on the adapter, as soon as any 3 of the 4 optional DVNs attest it.

JavaScript

```
function _checkVerifiable(
    UlnConfig memory _config,
    bytes32 _headerHash,
    bytes32 _payloadHash
) internal view returns (bool) {
    // iterate the required DVNs
    if (_config.requiredDVNCount > 0) {
        for (uint8 i = 0; i < _config.requiredDVNCount; ++i) {
            if (!_verified(_config.requiredDVNs[i], _headerHash,
                _payloadHash, _config.confirmations)) {
                // return if any of the required DVNs haven't signed
                return false;
            }
        }
    }
    if (_config.optionalDVNCount == 0) {
        // returns early if all required DVNs have signed and
        // there are no optional DVNs
        return true;
    }

    // then it must require optional validations
    uint8 threshold = _config.optionalDVNThreshold;
    for (uint8 i = 0; i < _config.optionalDVNCount; ++i) {
        if (_verified(_config.optionalDVNs[i], _headerHash,
            _payloadHash, _config.confirmations)) {
            // increment the optional count if the optional DVN has
            // signed
            threshold--;
            if (threshold == 0) {
                // early return if the optional threshold has hit
                return true;
            }
        }
    }

    // return false as a catch-all
    return false;
}
```

The weakness is that there is no required DVN that must sign every message. With at least one required DVN, that named operator must participate in every verification and therefore in every possible forgery, and an attacker cannot route around it. With an

all-optional policy, any 3-of-4 subset that an attacker can compromise or collude is sufficient, regardless of which specific operators they are, and the effective independence is lower than 4 distinct names suggests if any 3 of the operators share correlated infrastructure or dependencies.

This DVN policy is also mutable after deployment. It is changed by calling setConfig on the LayerZero endpoint, which the endpoint authorizes for the OApp itself or its delegate. The delegate is initialized to the MANAGER role, so the hot MANAGER key, not the DEFAULT_ADMIN_ROLE, currently holds the power to rewrite the DVN set and could point inbound verification at attacker controlled DVNs.

Recommendation

Consider promoting at least one operator into the required DVNs and reduce the optional threshold accordingly,

Status

Fixed, now enforces the Google DVN to be a required DVN, reducing the optional DVN threshold to a ⅔ configuration between LayerZero Labs, Nethermind and USDT0

[I15] OFTConfig sets the ETH side token name differing from BSC

Contract

OFTConfig.sol

Severity

Informational

Description

Description

OFTConfig.ethMainnet() and OFTConfig.sepolia() configure tokenName = "Lista DAO Token" and symbol = "LISTA". The deployment script DeployListaOFTv2 passes cfg.tokenName into ListaOFTv2.initialize, which forwards it to __OFT_init(name, ...) and __ERC20Permit_init(name). The BSC token defines its name as "Lista DAO", so the Ethereum side OFT is deployed with a name that does not match the asset it represents.

JavaScript

```
cfg.tokenName = "Lista DAO Token";  
cfg.symbol = "LISTA";
```

Impact

This produces a user facing metadata inconsistency across the mesh, since the ETH side OFT will display "Lista DAO Token" while the underlying asset is named "Lista DAO". In addition, the EIP-712 permit domain on the OFT is derived from this differing name. The domain remains internally consistent, so permit signatures created against this OFT will verify correctly, but it differs from any signatures users may construct expecting a "Lista DAO" domain. OFT bridging itself does not depend on the token name, so cross-chain transfers and user funds are not affected.

Recommendation	Set tokenName = "Lista DAO" in _ethMainnet() and _sepolia() so the OFT name matches the canonical token, or confirm that the rename to "Lista DAO Token" is intentional and document it so the metadata difference and the resulting permit domain are expected.
Status	Fixed, now switches the mainnet Lista LZ tokenName to match the BSC Lista token name of "Lista Dao".

[I16] Enforced LayerZero options are never configured during bring-up, leaving no destination gas floor for user sends

Contract	–
Severity	Informational
Description	Description The contracts expose setEnforcedOptions, which is gated to the MANAGER role and validated by _assertOptionsType3, but none of the deployment and configuration scripts ever call it. The bring-up sequence wires peers through SetPeer, libraries and DVNs through SetDVNConfig, and transfer limits through SetTransferLimit, yet enforced options are never set, so enforcedOptions[eid][msgType] remains empty. <pre> JavaScript /// @notice Sets enforced LayerZero options. Restricted to MANAGER. function setEnforcedOptions(EnforcedOptionParam[] calldata _enforcedOptions) public override onlyRole(MANAGER) { OAppOptionsType3Storage storage \$ = _getOAppOptionsType3Storage(); for (uint256 i = 0; i < _enforcedOptions.length; i++) { _assertOptionsType3(_enforcedOptions[i].options); \$.enforcedOptions[_enforcedOptions[i].eid][_enforcedOptions[i].msgType] = _enforcedOptions[i].options; } emit EnforcedOptionSet(_enforcedOptions); } </pre> Impact With enforced options empty, combineOptions returns only whatever options the caller supplies. The BridgeListaOFTV2 script happens to pass addExecutorLzReceiveOption(LZ_RECEIVE_GAS, 0) with a default of 300_000 gas, so it works, but a real user calling send with empty or insufficient extraOptions produces a message carrying too little destination gas. When that message is delivered, lzReceive invokes _credit and runs out of gas on the destination chain, leaving the message in a stuck or retryable state with funds stalled mid-bridge. Enforced options exist precisely to guarantee a minimum lzReceive gas floor regardless of what the caller provides, and that protection is currently absent.

Recommendation	Consider adding a SetEnforcedOptions script that configures enforced options per destination endpoint id for both the SEND and SEND_AND_CALL message types, and run it as part of the standard bring-up sequence alongside the peer, DVN, and limit configuration steps.
Status	Fixed, now introduces a SetEnforcedOptions.s.sol script that sets a type-3 enforced option with an executor lzReceive gas floor that uses the default 300000 equaling the value used in the BridgeListOFTV2.s.sol bridge test script for the SEND message type per dstEid.

[I17] _sorted orders DVN addresses but does not remove duplicates, causing setConfig to revert

Contract	OFTScriptBase.sol
Severity	Informational
Description	Description The OFTScriptBase._sorted function sorts a DVN address array in ascending order but does not remove duplicate entries, even though the ULN configuration requires the DVN array to be both sorted and unique. As long as the input contains no repeats the result is valid, so the issue is latent today. <pre> JavaScript /// @dev Ascending insertion sort (ULN requires sorted, unique DVN arrays). function _sorted(address[] memory arr) internal pure returns (address[] memory) { for (uint256 i = 1; i < arr.length; i++) { address key = arr[i]; uint256 j = i; while (j > 0 && uint160(arr[j - 1]) > uint160(key)) { arr[j] = arr[j - 1]; j--; } arr[j] = key; } return arr; } </pre> Impact If a future configuration passes a DVN address more than once, _sorted returns a sorted array that still contains the duplicate, and the subsequent call to endpoint.setConfig reverts because the ULN rejects non unique DVN arrays. This turns an easy to make configuration mistake into a failed setup transaction rather than a clean, self correcting input.

Recommendation	Consider extending the <code>_sorted</code> function to drop duplicate addresses after sorting so the returned array is both ordered and unique, matching what the ULN expects.
Status	Acknowledged, it is presumed that the trusted <code>MANAGER</code> holding delegate permissions will not set identical DVNs if <code>setConfig</code> is required to be invoked in the future.

[I18] Configuration scripts sanity checks

Contract	SetDVNConfig.sol & SetPeer.s.sol
Severity	Informational
Description	Description The deployment and configuration scripts wire the bridge across multiple steps and across two chains, which creates unsafe windows when the steps are run in the wrong order or only partially completed. 1. The first issue concerns the order of <code>setPeer</code> relative to <code>SetDVNConfig</code>. <code>SetDVNConfig</code> checks <code>isDefaultSendLibrary</code> and then overrides the default library and DVN set with the intended configuration, such as the 3-of-4 DVN policy. If <code>setPeer</code> is executed before <code>SetDVNConfig</code>, the route becomes live while the endpoint is still using LayerZero's default library and default DVN configuration instead of the chosen one. During that window, messages are verified under security settings that the protocol did not select. 2. The second issue concerns cross-chain configuration not being atomic. While one side is wired but the other is not, for example after BSC to ETH is configured but before ETH to BSC is configured, a send directed at the unconfigured side reaches a destination where <code>_getPeerOrRevert</code> reverts because no peer is set. The message does not fail permanently and no funds are lost, since it can be retried once the peer is configured, but the bridged value is left in limbo until both sides are fully wired. Consider: • Configuring the libraries and DVN settings with <code>SetDVNConfig</code> before calling <code>setPeer</code>, so a route is never live under default library and DVN settings.
Recommendation	• Do not allow users to bridge until both directions of the route are fully wired on both chains. • Additionally, since cross-chain configuration cannot be made atomic, launch the contracts in a paused state and only unpause after verifying, on both chains,

that libraries, DVNs, enforced options, transfer limits, and peers are all correctly set.

Status

Fixed, now includes a comprehensive step-by-step checklist during the launch of the Lista token and bridge configurations. The suggested pause state will also be implemented to ensure correctness of the bridge configurations before unpause.